Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Операционные среды и системное программирование»

ОТЧЕТ
к лабораторной работе №6
на тему:
«ЭЛЕМЕНТЫ СЕТЕВОГО ПРОГРАММИРОВАНИЯ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

(дата, подпись студента)

Проверил ассистент каф. информатики
ГРИЦЕНКО Никита Юрьевич

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1 Индивидуальное задание 3
2 Выполнение работы 4
Вывод 6
Список использованных источников 7
Приложение А (справочное) Исходный код 8

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Написать программу в соответствии с вариантом задания, реализовав взаимодействие через сеть TCP/IP, и подготовить средства сборки и демонстрации работы.

Сервер/клиент пользовательской сетевой службы: необходимо реализовать службу обработки данных с установкой сервера на заданный порт, подключением клиента, передачей запроса, обработкой на стороне сервера, возвратом результата и базовой обработкой ошибок.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для лабораторной работы был выбран наиболее компактный вариант пользовательской сетевой службы: обработка числовых данных по протоколу TCP. Реализация состоит из двух программ: сервера number_server и клиента number_client. Клиент передаёт серверу целое число, а сервер вычисляет квадрат и куб этого числа и возвращает ответ в текстовом формате.

Сервер создаёт сокет вызовом socket(), разрешает повторное использование адреса через setsockopt(), после чего выполняет bind() к указанному порту и переводит сокет в режим ожидания соединений при помощи listen() [1]. Далее в цикле вызывается accept(), принимается клиентское соединение, считывается строка с числом, выполняется её разбор функцией strtol() и формируется ответ.

Клиент использует getaddrinfo() для разрешения адреса сервера и затем перебирает найденные варианты, пока не удастся успешно выполнить connect() [2]. После установления соединения клиент отправляет число строкой, получает ответ с помощью recv() и выводит его пользователю. Если число передано в неверном формате, сервер возвращает сообщение ERROR Invalid integer.

Для сборки и демонстрации создан Makefile с целями all, run-server, run-client и clean. Это позволяет быстро собрать обе программы и запустить типовой сценарий взаимодействия без ручного ввода длинных команд.

На рисунке 2.1 показан журнал сервера: после запуска он устанавливается на порт 5555 и обслуживает два запроса от локального клиента. В журнале видны адрес клиента, порт соединения и сформированный ответ, что подтверждает корректную работу серверной части и прикладного протокола.

```
[koxan@victusMonjaro LR6]$ make run-server
./number_server 5555
Server started on port 5555
Service: number processing (square and cube)
Client 127.0.0.1:48760 -> OK value=7 square=49 cube=343
```

Рисунок 2.1 – Журнал работы TCP-сервера

На рисунке 2.2 приведены ответы клиента для корректного запроса. При отправке числа 7 клиент получает строку: OK value=7 square=49 cube=343.

```
[koxan@victusMonjaro LR6]$ make run-client
./number_client 127.0.0.1 5555 7
OK value=7 square=49 cube=343
```

Рисунок 2.2 – Результат работы программы при корректном запросе

При передаче строки “a” сервер возвращает сообщение об ошибке, это продемонстрированно на рисунке 2.3.

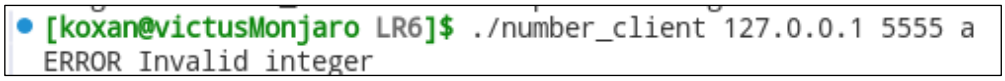


Рисунок 2.3 – Результат работы программы при передаче строки вместо числа

При передаче нечисловой строки, например “a”, сервер возвращает сообщение ERROR Invalid integer, а клиент выводит его на экран, что видно на рисунке 2.3. Это демонстрирует, что прикладной протокол включает в себя не только успешную обработку, но и информативные коды ошибок. В целом программа соответствует варианту «Сервер/клиент пользовательской сетевой службы» с функциональностью обработки числовых данных. Сервер построен по классической схеме: socket → bind → listen → accept → recv/send → close. Он работает в бесконечном цикле, обрабатывая по одному соединению за раз. Для корректного завершения при получении сигнала SIGINT используется флаг volatile sig_atomic_t keep_running, который проверяется в основном цикле и после выхода из accept при ошибке EINTR. Клиент использует getaddrinfo для разрешения имени хоста и номера порта, что делает его устойчивым как к IPv4, так и к IPv6.

После установления соединения клиент отправляет строку, полученную из аргумента командной строки, и ждёт ответа. Ответ считывается построчно с помощью вспомогательной функции read_line, которая имитирует чтение до символа '\n', что удобно для текстового протокола. Makefile включает цели all (сборка сервера и клиента), run-server (запуск сервера на порту 5555), run-client (запуск клиента с параметрами 127.0.0.1 5555 7) и clean. Для проверки ошибочного ввода можно вручную выполнить команду ./number_client 127.0.0.1 5555 a. Таким образом, программа полностью удовлетворяет индивидуальному заданию и демонстрирует практическое освоение сетевого программирования на сокетах TCP/IP.

ВЫВОД

В результате выполнения лабораторной работы были достигнуты цели лабораторной работы. Изучены основы построения и функционирования сетей, стеков протоколов (TCP/IP), а также программный интерфейс сокетов в Unix-системах. Освоены основные системные вызовы: socket(), bind(), listen(), accept(), connect(), send(), recv(), а также getaddrinfo() для разрешения имён. Реализована пользовательская сетевая служба – сервер и клиент, взаимодействующие по протоколу TCP. Сервер обрабатывает запросы на вычисление квадрата и куба целого числа, возвращая результат в текстовом формате; клиент отправляет число и выводит ответ. Предусмотрена базовая валидация входных данных и обработка ошибок (некорректное число, проблемы с соединением). Сервер может корректно завершаться по сигналу SIGINT без утечки ресурсов. Для автоматизации сборки и тестирования создан Makefile с целями all, run-server, run-client, clean. Проведена демонстрация работы службы как с корректными, так и с некорректными запросами. Таким образом, все задачи лабораторной работы выполнены в полном объёме, приобретены практические навыки проектирования, реализации и отладки программ, взаимодействующих через сеть TCP/IP с использованием сокетов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Linux manual page socket(2) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man2/socket.2.html. – Дата доступа: 11.02.2026.

[2] Linux manual page getaddrinfo(3) [Электронный ресурс]. – Режим доступа: https://man7.org/linux/man-pages/man3/getaddrinfo.3.html. – Дата доступа: 11.02.2026.

ПРИЛОЖЕНИЕ А
(справочное)
Исходный код

Листинг А.1 – Реализация TCP-сервера

```
#define _POSIX_C_SOURCE 200809L

#include <arpa/inet.h>
#include <errno.h>
#include <netinet/in.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define BACKLOG 5
#define BUFFER_SIZE 256

static volatile sig_atomic_t keep_running = 1;

static void handle_sigint(int sig)
{
    (void)sig;
    keep_running = 0;
}

static int read_line(int fd, char *buffer, size_t size)
{
    size_t total = 0;

    while (total + 1 < size)
    {
        char ch;
        ssize_t n = recv(fd, &ch, 1, 0);

        if (n == 0)
            return 0;
        if (n < 0)
            return -1;

        buffer[total++] = ch;
        if (ch == '\n')
            break;
    }

    buffer[total] = '\0';
    return 1;
}

int main(int argc, char *argv[])
{
    int server_fd = -1;
    int client_fd = -1;
    int port;
    struct sockaddr_in server_addr;

    if (argc != 2)
    {
```

```
        fprintf(stderr, "Usage: %s <port>\n", argv[0]);
        return EXIT_FAILURE;
    }

    port = atoi(argv[1]);
    if (port <= 0 || port > 65535)
    {
        fprintf(stderr, "Invalid port: %s\n", argv[1]);
        return EXIT_FAILURE;
    }

    signal(SIGINT, handle_sigint);

    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0)
    {
        perror("socket");
        return EXIT_FAILURE;
    }

    {
        int opt = 1;
        if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt))
< 0)
        {
            perror("setsockopt");
            close(server_fd);
            return EXIT_FAILURE;
        }
    }

    memset(&server_addr, 0, sizeof(server_addr));
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    server_addr.sin_port = htons((unsigned short)port);

    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr))
< 0)
    {
        perror("bind");
        close(server_fd);
        return EXIT_FAILURE;
    }

    if (listen(server_fd, BACKLOG) < 0)
    {
        perror("listen");
        close(server_fd);
        return EXIT_FAILURE;
    }

    printf("Server started on port %d\n", port);
    printf("Service: number processing (square and cube)\n");

    while (keep_running)
    {
        struct sockaddr_in client_addr;
        socklen_t client_len = sizeof(client_addr);
        char buffer[BUFFER_SIZE];
        char response[BUFFER_SIZE];
        char client_ip[INET_ADDRSTRLEN];
        char *endptr;
        long value;
        long long square;
```

```
        long long cube;
        int status;

        client_fd = accept(server_fd, (struct sockaddr *)&client_addr,
&client_len);
        if (client_fd < 0)
        {
            if (errno == EINTR && !keep_running)
                break;
            perror("accept");
            continue;
        }

        if (inet_ntop(AF_INET, &client_addr.sin_addr, client_ip,
sizeof(client_ip)) == NULL)
            strcpy(client_ip, "unknown");

        status = read_line(client_fd, buffer, sizeof(buffer));
        if (status <= 0)
        {
            if (status < 0)
                perror("recv");
            close(client_fd);
            client_fd = -1;
            continue;
        }

        value = strtol(buffer, &endptr, 10);
        while (*endptr == ' ' || *endptr == '\t')
            endptr++;

        if (endptr == buffer || (*endptr != '\n' && *endptr != '\0'))
        {
            snprintf(response, sizeof(response), "ERROR Invalid integer\n");
        }
        else
        {
            square = (long long)value * value;
            cube = square * value;
            snprintf(response, sizeof(response),
                     "OK value=%ld square=%lld cube=%lld\n",
                     value, square, cube);
        }

        if (send(client_fd, response, strlen(response), 0) < 0)
            perror("send");
        else
            printf("Client %s:%d -> %s",
                   client_ip,
                   ntohs(client_addr.sin_port),
                   response);

        close(client_fd);
        client_fd = -1;
    }

    if (client_fd >= 0)
        close(client_fd);
    close(server_fd);
    printf("Server stopped\n");

    return EXIT_SUCCESS;
}
```

Листинг А.2 – Реализация TCP-клиента

```
#define _POSIX_C_SOURCE 200809L

#include <arpa/inet.h>
#include <netdb.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

#define BUFFER_SIZE 256

static int read_line(int fd, char *buffer, size_t size)
{
    size_t total = 0;

    while (total + 1 < size)
    {
        char ch;
        ssize_t n = recv(fd, &ch, 1, 0);

        if (n == 0)
            break;
        if (n < 0)
            return -1;

        buffer[total++] = ch;
        if (ch == '\n')
            break;
    }

    buffer[total] = '\0';
    return (int)total;
}

int main(int argc, char *argv[])
{
    int sockfd = -1;
    struct addrinfo hints;
    struct addrinfo *result = NULL;
    struct addrinfo *rp;
    char request[BUFFER_SIZE];
    char response[BUFFER_SIZE];
    int status;

    if (argc != 4)
    {
        fprintf(stderr, "Usage: %s <host> <port> <integer>\n", argv[0]);
        return EXIT_FAILURE;
    }

    snprintf(request, sizeof(request), "%s\n", argv[3]);

    memset(&hints, 0, sizeof(hints));
    hints.ai_family = AF_INET;
    hints.ai_socktype = SOCK_STREAM;

    status = getaddrinfo(argv[1], argv[2], &hints, &result);
    if (status != 0)
    {
        fprintf(stderr, "getaddrinfo: %s\n", gai_strerror(status));
```

```
        return EXIT_FAILURE;
    }

    for (rp = result; rp != NULL; rp = rp->ai_next)
    {
        sockfd = socket(rp->ai_family, rp->ai_socktype, rp->ai_protocol);
        if (sockfd < 0)
            continue;

        if (connect(sockfd, rp->ai_addr, rp->ai_addrlen) == 0)
            break;

        close(sockfd);
        sockfd = -1;
    }

    freeaddrinfo(result);

    if (sockfd < 0)
    {
        perror("connect");
        return EXIT_FAILURE;
    }

    if (send(sockfd, request, strlen(request), 0) < 0)
    {
        perror("send");
        close(sockfd);
        return EXIT_FAILURE;
    }

    status = read_line(sockfd, response, sizeof(response));
    if (status < 0)
    {
        perror("recv");
        close(sockfd);
        return EXIT_FAILURE;
    }

    printf("%s", response);

    close(sockfd);
    return EXIT_SUCCESS;
}
```

Листинг А.3 – Файл сборки проекта

```
CC = gcc
CFLAGS = -Wall -Wextra -Wpedantic -std=c11

SERVER = number_server
CLIENT = number_client

all: $(SERVER) $(CLIENT)

$(SERVER): server.c
    $(CC) $(CFLAGS) server.c -o $(SERVER)

$(CLIENT): client.c
    $(CC) $(CFLAGS) client.c -o $(CLIENT)

run-server: $(SERVER)
    ./$(SERVER) 5555
```

```
run-client: $(CLIENT)
    ./$(CLIENT) 127.0.0.1 5555 7

clean:
    rm -f $(SERVER) $(CLIENT)
```